



3.2.3 推导与语法树

对于推导:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$$

它产生句子的方式很不直观, 看起来十分困难。

语法树是推导的图形表示, 它的表示很直观, 并且同时反映语言结构的实质和推导过程。

定义3.5 对CFG G 的句型, 语法树被定义为具有下述性质的一棵树。

- (1) 根由开始符号所标记;
- (2) 每个叶子由一个终结符, 或一个非终结符, 或 ε 标记;
- (3) 每个内部结点由一个非终结符标记;
- (4) 若 A 是某内部节点的标记, 且 $X_1, X_2, \dots, X_n$ 是该节点从左到右所有孩子的标记, 则 $A \rightarrow X_1X_2 \dots X_n$ 是一个产生式。若 $A \rightarrow \varepsilon$, 则标记为 A 的结点可以仅有一个标记为 ε 的孩子。





语法树与语言 and 文法的关系:

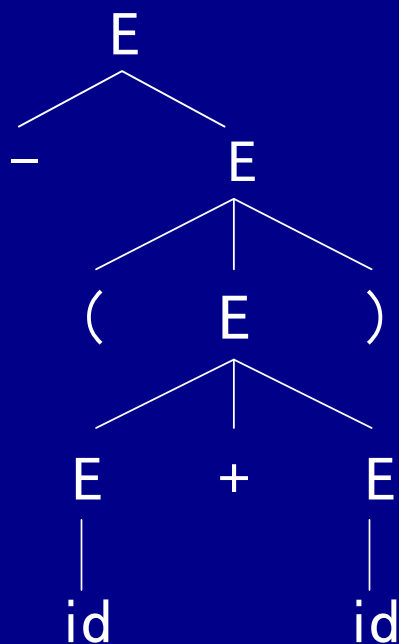
- **与文法的关系**: 每一直接推导 (每个产生式), 对应一棵仅有父子关系的子结构, 即产生式左部非终结符“长出”右部的孩子;
- **与语言的关系**: 语法树的叶子, 从左到右构成 G 的一个句型。若叶子仅由终结符标记, 则构成一个句子。



例3.5 再考察 $-(id+id)$ 的推导过程。

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

用语法树的方式如下:

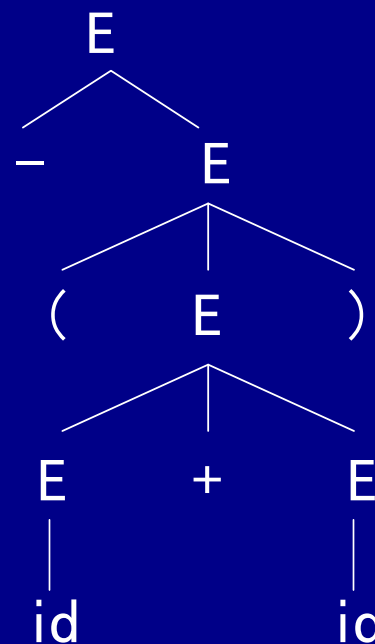


$E \rightarrow -E$

$E \rightarrow (E)$

$E \rightarrow E + E$

$E \rightarrow id$



1. 最左推导和最右推导的中间过程对应的语法树可能不同, 因为句型不同: $-(id+E)$ 或 $-(E+id)$
2. 但是最终的语法树相同, 因为最终是同一个句子: $-(id+id)$
3. 语法树既反映了产生句型的推导过程, 又反映了句型的结构。



更多的情况下，仅关注句型结构，而忽略推导过程。

定义3.6 对CFG G 的句型，表达式的抽象语法树被定义为具有下述性质的一棵树：

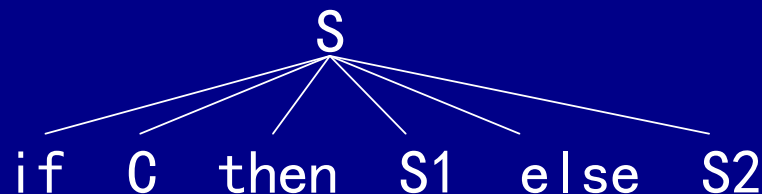
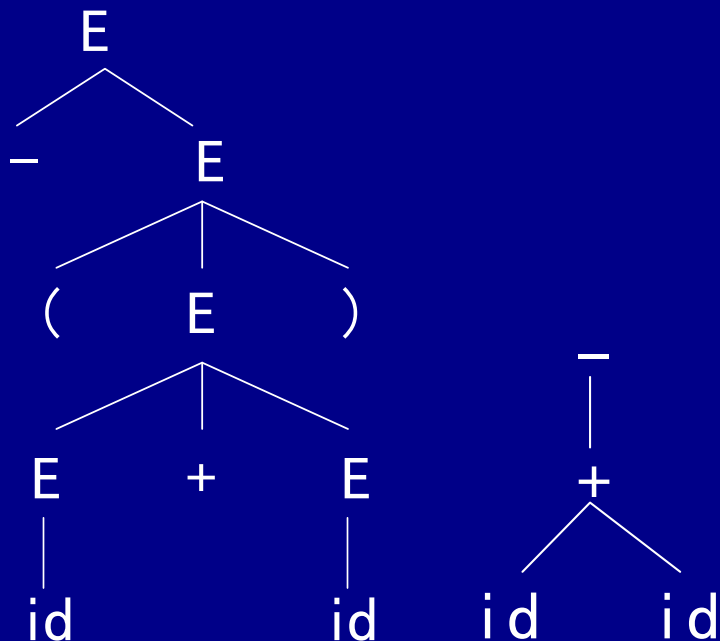
- (1) 根与内部节点由表达式中的操作符标记；
- (2) 叶子由表达式中的操作数标记；
- (3) 用于改变运算优先级和结合性的括弧，被隐含在语法树的结构中。

实质上，语法树与抽象语法树的最根本区别在于它们的内部节点（包括根节点）：

- 语法树的内部节点是非终结符；
- 抽象语法树的内部节点是操作符（运算符）；
- 或者说抽象语法树中省略了反映分析过程的非终结符。



例3.6 句子 -(id+id) 和句型 if C then s1 else s2 ;



语法树：左部非终结符“产生”右部文法符号序列；

抽象语法树：操作符（运算）“作用于”操作数（运算对象）；

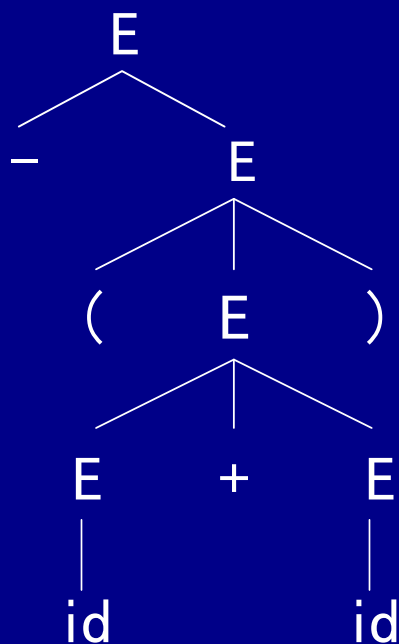
习惯上：它们分别被称为具体语法树和抽象语法树。



例3.5 再考察 $-(id+id)$ 的推导过程。

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

用语法树的方式如下：

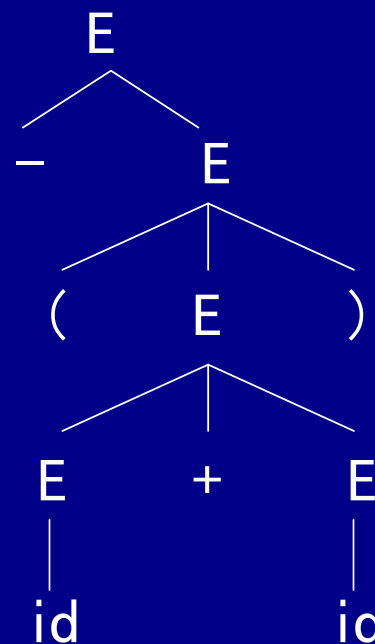


$E \rightarrow -E$

$E \rightarrow (E)$

$E \rightarrow E + E$

$E \rightarrow id$



1. 最左推导和最右推导的中间过程对应的语法树可能不同，因为句型不同： $-(id+E)$ 或 $-(E+id)$
2. 但是最终的语法树相同，因为最终是同一个句子： $-(id+id)$
3. 语法树既反映了产生句型的推导过程，又反映了句型的结构。

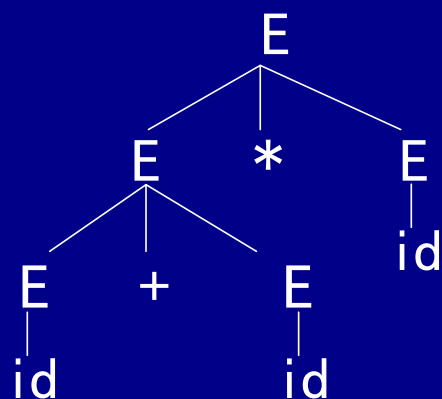


3.2.4 二义性与二义性的消除

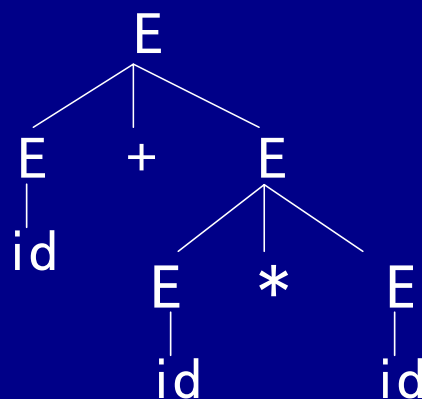
1. 二义性 (歧义, Ambiguity)

问题：一个句子可能对应多于一棵语法树

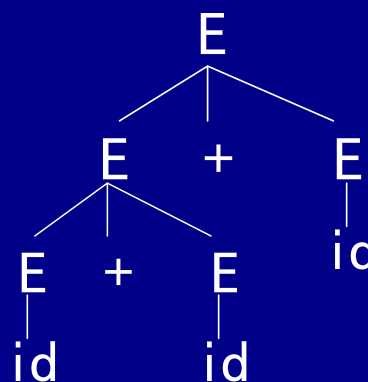
例3.7 句子 id+id*id 和 id+id+id 可能的语法树：



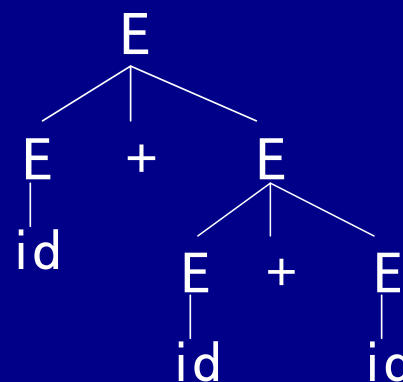
(a) 先进行+运算



(b) 先进行*运算



(c) +左结合



(d) +右结合

(G3.2) $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid id$



定义3.7 (文法二义性) 若文法G对同一句子产生不止一棵语法树，则称文法G是二义的。或者，若文法G对同一个句子存在两个不同的最左（右）推导，则称文法G是二义的。 ■

原因：在产生句子的过程中某些直接推导有多于一种选择

- 一个句子有多于一棵语法树，仅与文法和句子有关，与采用的推导方法无关；
- 表达式文法存在二义性通常是由于文法中缺少对文法符号优先级和结合性的规定。



语言的二义性：如果产生上下文无关语言的每一个文法都是二义的，则说此语言是先天二义的。

可能有两个不同的文法 G 和 G' ，其中 G 是二义的，但是却有 $L(G)=L(G')$ ，也就是说，这两个文法所产生的语言是相同的。

问题1：是否存在一个算法，使得对于任给的一个上下文无关文法 G ，该算法都能够计算出 G 是否是一个二义文法？

答：不存在这样的算法。

即，**上下文无关文法的二义性问题是不可判定的。**

问题2：如何证明一个CFG是二义文法？

“悬空 (dangling) else” 问题

3.2.4.1 二义性 (续2)



$S \rightarrow \text{if } C \text{ then } S$ (1)

$\quad | \text{if } C \text{ then } S \text{ else } S$ (2)

$\quad | \text{id} := E$ (3)

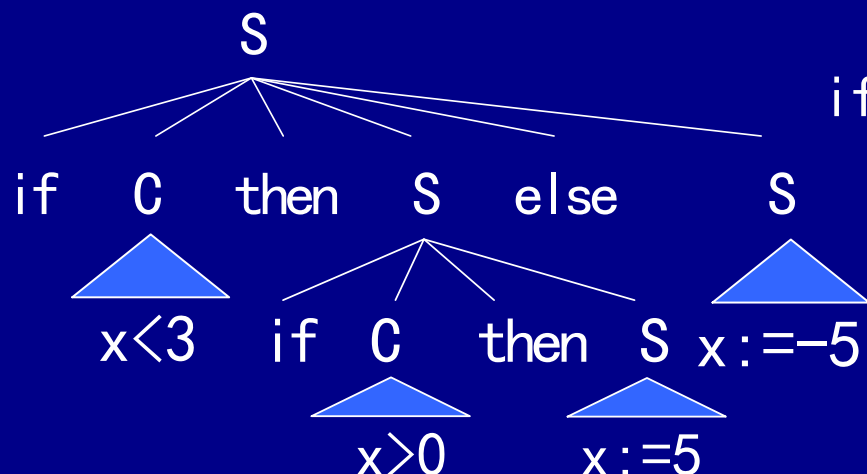
(G3.3)

$C \rightarrow E = E \mid E < E \mid E > E$ (4)... (6)

$E \rightarrow E + E \mid - E \mid \text{id} \mid n$ (7)... (10)

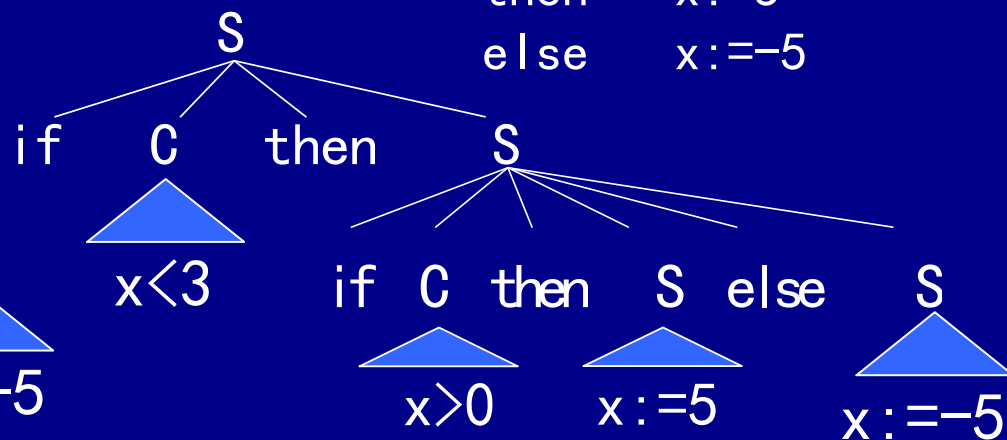
例3.8 条件语句 $\text{if } x < 3 \text{ then if } x > 0 \text{ then } x := 5 \text{ else } x := -5$

if $x < 3$
then if $x > 0$ then $x := 5$
else $x := -5$



(a) else与离它远的then匹配

if $x < 3$
then if $x > 0$
then $x := 5$
else $x := -5$



(b) else与离它近的then匹配



2. 二义性的消除

再分析 id+id*id 和 id+id+id:

文法二义不能说明程序设计语言是二义。程序设计语言不能二义。

消除语言二义的方法:

① 改写二义文法为非二义文法;

② 规定二义文法中符号的优先级和结合性, 使仅产生一棵语法树。

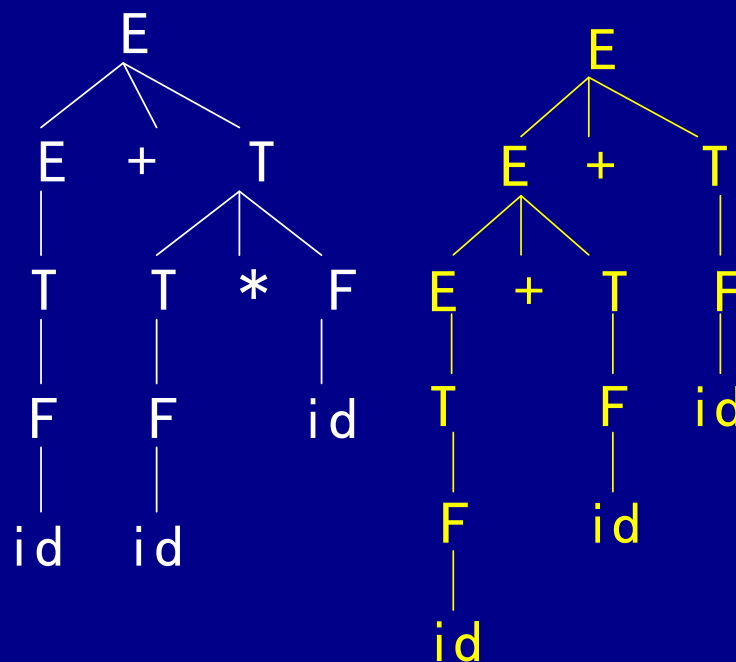
<1> 改写二义文法为非二义文法

例3.9 与G3.2等价的非二义文法:

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid -F \mid id
 \end{aligned}
 \quad (G3.4)$$

$$\begin{aligned}
 E &\rightarrow E + E \\
 &\mid E * E \\
 &\mid (E) \\
 &\mid -E \\
 &\mid id
 \end{aligned}
 \quad (G3.2)$$

问题: 如何将二义文法改写为非二义文法?





<1> 改写二义文法为非二义文法 (续1)

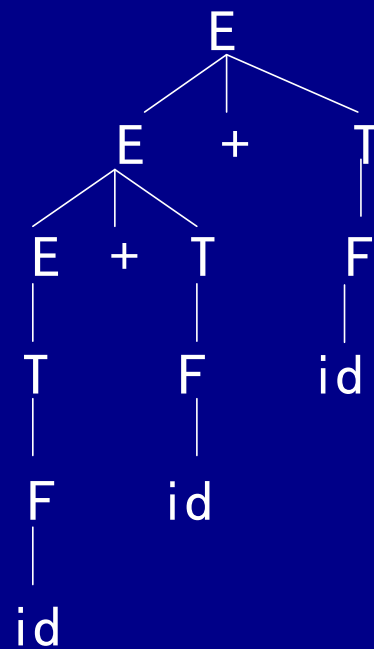
可以看出:

1. 新引入的非终结符, 限制了每一步直接推导均有唯一选择;
2. 非终结符的引入, 增加了推导步骤 (语法树增高);
3. 最终语法树的形状, 仅与文法有关, 而与推导方法无关;
4. 越接近S的文法符号的优先级越低。 (如 $E \rightarrow E+T$)。
5. 对于 $A \rightarrow \alpha A \beta$, 若A在终结符左边出现 (即终结符在 β 中), 则A产生式具有左结合性质。

改写二义文法的关键步骤:

1. 引入一个新的非终结符, 增加一个子结构并提高一级优先级;
2. 递归非终结符在终结符左边, 运算具有左结合性, 否则具有右结合性。

例3.10 改写二义文法G3.2为G3.4 ;



改写二义文法:



1. 引入一个新的非终结符, 增加一个子结构并提高一级优先级;
2. 递归非终结符在终结符左边, 运算具有左结合性, 否则具有右结合性。

① 优先级: [+]	② 结合性: 左结合	[+]	[*]	[-]	$E \rightarrow E + E$	(1)
					$E \rightarrow E * E$	(2)
					$E \rightarrow (E)$	(3)
					$E \rightarrow -E$	(4)
					$E \rightarrow id$	(5)

③ 非终结符与运算:

$E: +$	(E产生式, 左递归)	$E \rightarrow E + T$		T
$T: *$	(T产生式, 左递归)	$T \rightarrow T * F$		F
$F: -$	(F产生式, 右递归)	$F \rightarrow -F$		G
$G: (), id$	(G产生式)	$G \rightarrow (E)$		id

$E \rightarrow E + T$		T
$T \rightarrow T * F$		F
$F \rightarrow -F$		$(E) \mid id$

再讨论“悬空else”问题

<1> 改写二义文法为非二义文法 (续2)



问题： if-then-else和if-then: 在一个复合if语句中，可能then多于else，使得else不知与哪个then结合。

改写原则： else与其左边最靠近的then结合。

改写方法： 是将S分为完全匹配 (MS) 和不完全匹配 (UMS) 两类。

$S \rightarrow MS$
 $\quad \mid UMS$

$MS \rightarrow \text{if } C \text{ then } MS \text{ else } MS$
 $\quad \mid \text{id} := E$

$UMS \rightarrow \text{if } C \text{ then } S$
 $\quad \mid \text{if } C \text{ then } MS \text{ else } UMS$

$C \rightarrow E = E \mid E < E \mid E > E$

$E \rightarrow E + T \mid T$

$T \rightarrow -T \mid \text{id} \mid n$

(1) $S \rightarrow \text{if } C \text{ then } S$

(2) $\quad \mid \text{if } C \text{ then } S \text{ else } S$

(3) $\quad \mid \text{id} := E$ (G3.3)

(4) $C \rightarrow E = E \mid E < E \mid E > E$

(5) $E \rightarrow E + E \mid -E \mid \text{id} \mid n$

(6) (G3.5)

(7)... (9)

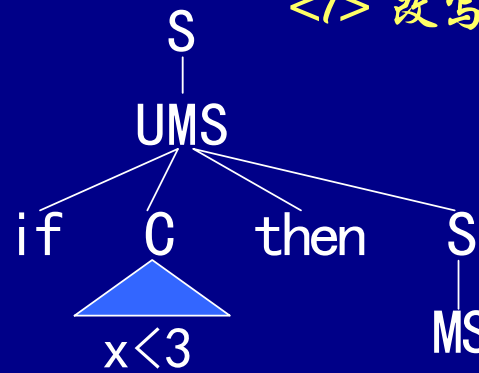
(10)... (11)

(12)... (14)

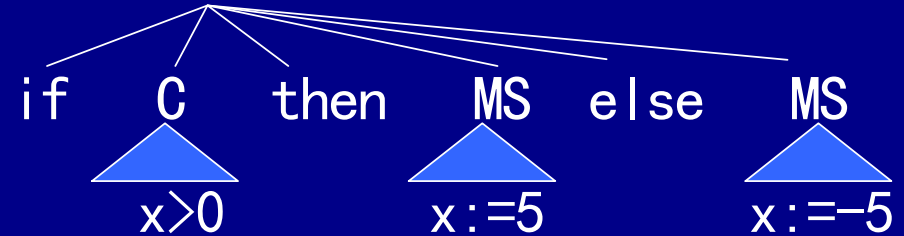


<1> 改写二义文法为非二义文法

if x<3
then if x>0
then x:=5
else x:=-5



if x<3
then **不可能!** then x:=5
else x:=-5



S → MS
| UMS
MS → if C then MS else MS
| id := E
UMS → if C then S
| if C then MS else UMS
C → E = E | E < E | E > E
E → E + T | T
T → -T | id | n

(1)
(2)
(3)
(4) if C then MS else MS
(5)
(6) if C then MS else MS
(7)... (9)
(10)... (11) (G3.5)
(12)... (14)

```

graph TD
    S --> MS
    MS --> if1[if]
    MS --> C1[C]
    MS --> then1[then]
    MS --> else1[else]
    MS --> MS1[MS]
    C1 --> xlt3[x<3]
    then1 --> MS2[MS]
    MS2 --> x5[x:=5]
    else1 --> MS3[MS]
    MS3 --> xneg5[x:=-5]
  
```

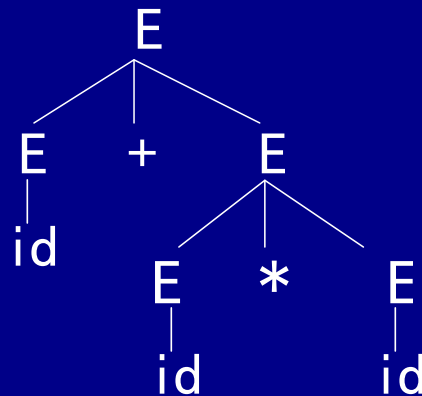


<2> 为文法符号规定优先级和结合性

二义文法的优点:

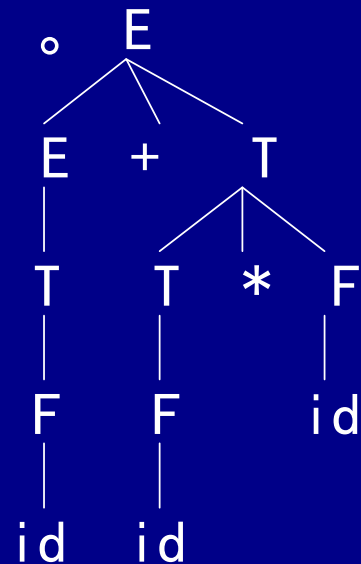
- ① 比非二义文法容易理解;
- ② 分析效率高 (语法树低, 直接推导步骤少)。

对于: $id + id * id$

$$\begin{array}{l} E \rightarrow E + E \\ \quad | E * E \\ \quad | - E \\ \quad | (E) \\ \quad | id \end{array}$$


$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid -F \mid id$$


为二义文法规定优先级和结合性 (YACC的方法):

```
%left '+'
```

```
%left '*'
```

```
%right '-'
```

```
E : E '+' E | E '*' E | '-' E | '(' E ')' | id
```




<3> 修改语言的语法 (表现形式被改变)

① Ada中用end if结束条件语句，于是有：

```
if      x<3
then    if x>0
        then x:=5;
        end if;
else    x:=-5;
end if;
```

```
if      x<3
then    if x>0
        then x:=5;
        else x:=-5;
        end if;
end if;
```

② 给表达式加括号，如Pascal中逻辑和算术运算：

$(a+b) > (c*d)$



3.3 语言与文法简介

文法的重要作用：

1. 给出精确、易于理解的语言结构说明；
2. 以文法为基础的语言，便于加入新的、或修改、删除旧的语言结构；
3. 有些类别的文法，可以自动生成高效的分析器。

本节从理论的角度对文法进行简单地讨论。讨论建立在形式语言与自动机的理论之上，且仅引用结论而忽略数学的证明，有兴趣的同学可以参阅相关文献。

希望通过本节的讨论，对文法的分类和它们在编译器构造中的作用有一定的了解。



3.3.1 正规式与上下文无关文法

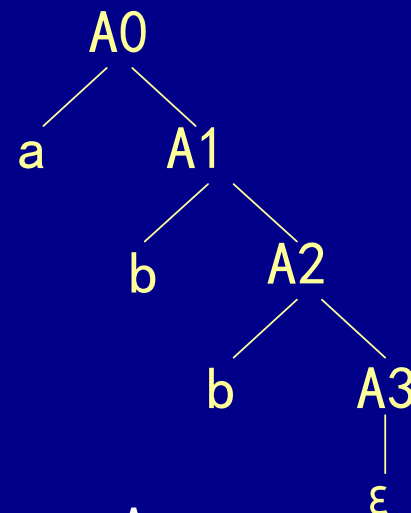
<1> 正规式到CFG的转换

推论3.1 正规式所描述的语言结构均可用CFG描述，反之不一定。 ■

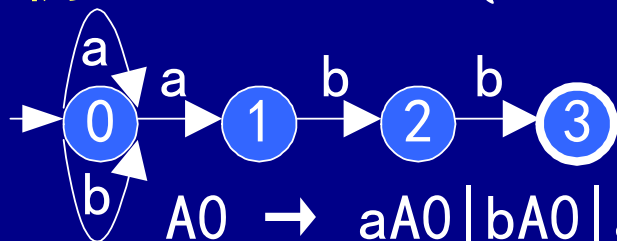
从正规式到CFG的对应关系：

1. 构造正规式的NFA;
2. 若0为初态，则A0为开始符号;
3. 对于 $\text{move}(i, a)=j$ ，引入产生式 $A_i \rightarrow aA_j$;
4. 对于 $\text{move}(i, \varepsilon)=j$ ，引入产生式 $A_i \rightarrow A_j$;
5. 若i是终态，则引入产生式 $A_i \rightarrow \varepsilon$ 。

分别产生abb的语法树：



例3.11 从正规式 $r=(a|b)^*abb$ 的NFA构造CFG:



$A_0 \rightarrow aA_0 | bA_0 | aA_1$

$A_1 \rightarrow bA_2$

$A_2 \rightarrow bA_3$

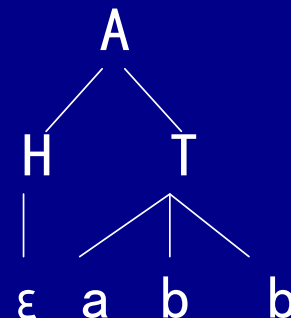
$A_3 \rightarrow \varepsilon$

经验的方法：

$A \rightarrow HT$

$H \rightarrow \varepsilon | Ha | Hb$

$T \rightarrow abb$





<2> 为什么用正规式而不用CFG描述词法

1. 词法规则简单，用正规式描述已足够；
2. 正规式的表示比CFG更直观、简洁、易于理解；
3. 有限自动机的构造比下推自动机简单，且分析效率高；
4. 区分词法和语法，为编译器前端的模块划分提供方便。

贯穿词法分析和语法分析始终的思想：

1. 一般情况下，正规式适合描述线性结构，如标识符、关键字、注释等；
2. CFG适合描述具有嵌套（层次）性质的非线性结构，如不同结构的句子if-then-else、while-do等。
3. 语言的描述和语言的识别是表示一个语言的两个不同侧面，二者缺一不可；（语言、文法、自动机）
4. 用正规式和CFG描述的语言，对应的识别方法（自动机）不同；



3.3.2 上下文有关语言

(Context Sensitive Language, CSL)

程序设计语言中除了CFG可以描述的结构之外，还有一些是CFG无法描述的所谓**上下文有关的结构**。

典型的这类语言结构包括：变量的声明与引用、过程调用时形参与实参的一致性检查等。

描述它们的文法被称为上下文有关文法 (Context Sensitive Grammar, CSG)。



例3.12 不能用CFG描述的语言:

$L1 = \{\omega c \omega \mid \omega \in (a b)^*\}$	(标识符声明与引用一致性的抽象)
$L2 = \{a^n b^m c^n d^m \mid n \geq 1 \text{ 和 } m \geq 1\}$	(形参与实参一致性的抽象)
$L3 = \{a^n b^n c^n \mid n \geq 1\}$	(计数问题的抽象)

相近的CFL:

$L1' = \{\omega c \omega^r \mid \omega \in (a b)^*\}$	$(S \rightarrow aSa \mid bSb \mid c)$
$L2' = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$	$(S \rightarrow aSd \mid aAd \quad A \rightarrow bAc \mid bc)$
$L2'' = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}$	$(S \rightarrow AB \quad A \rightarrow aAb \mid ab \quad B \rightarrow cBd \mid cd)$
$L3' = \{a^m b^m c^n \mid m, n \geq 1\}$	$(S \rightarrow AC \quad A \rightarrow aAb \mid ab \quad C \rightarrow cC \mid c)$



计数问题

$L3 = \{a^n b^n c^n \mid n \geq 1\}$ CSL

$L3' = \{a^m b^m c^n \mid m, n \geq 1\}$ CFL $S \rightarrow AC \quad A \rightarrow aAb \mid ab \quad C \rightarrow cC \mid c$

$L3'' = \{a^k b^m c^n \mid k, m, n \geq 1\}$ 正规集 $a^+ b^+ c^+$

命题: $L3'$ 不是正规集, 因为构造不出可以识别 $L3'$ 的 DFA。 ■

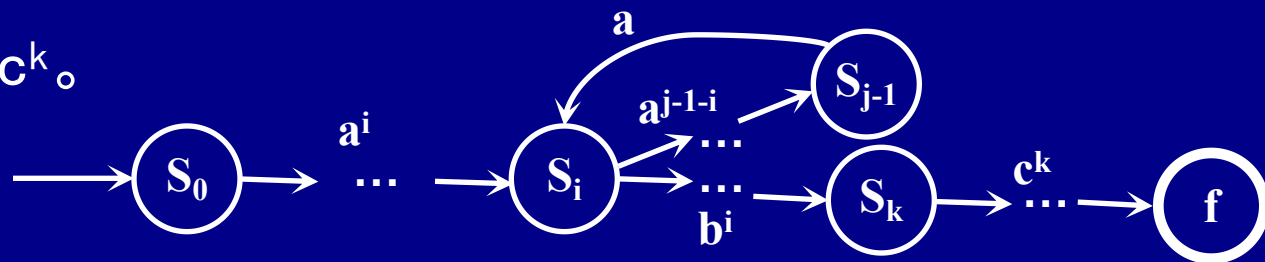
证明: (反证)

假设 $L3'$ 是正规集, 即可构造 n 个状态的 DFA D , 它接受 $L3'$;

考察 D 读完 $\varepsilon, a, aa, \dots, a^n$, 分别到达 $S_0, S_1, \dots, S_n$, 共有 $n+1$ 个状态。

根据鸽巢原理, 序列中至少有两个状态相同, 设 $S_i = S_j$ ($j > i$), 因为 $a^i b^i c^k \in L3'$

所以存在路径 $a^i b^i c^k$ 。



但是 D 中也有路径 $a^j b^i c^k$, 矛盾。故 $L3'$ 不是正规集。 ■



3.3.3 形式语言与自动机简介

定义3.8 若文法 $G=(N, T, P, S)$ 的每个产生式 $\alpha \rightarrow \beta$ 中, 均有 $\alpha \in (N \cup T)^*$, 且至少含有一个非终结符, $\beta \in (N \cup T)^*$, 则称 G 为 0 型文法。(Chomsky 文法分类)

对 0 型文法施加以下第 i 条限制, 即得到 i 型文法。

1. G 的任何产生式 $\alpha \rightarrow \beta$ ($\alpha \rightarrow \varepsilon$ 除外) 满足 $|\alpha| \leq |\beta|$;
2. G 的任何产生式形如 $A \rightarrow \beta$, 其中 $A \in N$, $\beta \in (N \cup T)^*$;
3. G 的任何产生式形如 $A \rightarrow a$ 或者 $A \rightarrow aB$ (或者 $A \rightarrow Ba$), 其中 A 和 $B \in N$, $a \in T$ 。

文法、语言与自动机

文 法	语 言	自 动 机
短语文法 (0型)	短语结构语言	图灵机
CSG (1型)	CSL	线性界限自动机
CFG (2型)	CFL	下推自动机
正规文法 (3型)	正规集	有限自动机

再考察L3:

$$L3 = \{a^n b^n c^n \mid n \geq 1\}$$

$$L3' = \{a^m b^m c^n \mid m, n \geq 1\} \quad (S \rightarrow AC \quad A \rightarrow aAb \mid ab \quad C \rightarrow cC \mid c)$$

$$L3'' = \{a^k b^m c^n \mid k, m, n \geq 1\} \quad (a^+ b^+ c^+)$$

例3.15 L3可用下述CSG描述:

$$S \rightarrow aSBC \quad (1)$$

$$S \rightarrow aBC \quad (2)$$

$$CB \rightarrow BC \quad (3)$$

$$aB \rightarrow ab \quad (4)$$

$$bB \rightarrow bb \quad (5)$$

$$bC \rightarrow bc \quad (6)$$

$$cC \rightarrow cc \quad (7)$$

句子 $a^k b^k c^k$ 的推导:

$$S \Rightarrow \dots \Rightarrow a^{k-1} S (BC)^{k-1} \quad (1)$$

$$\Rightarrow a^k (BC)^k \quad (2)$$

$$\Rightarrow \dots \Rightarrow a^k B^k C^k \quad (3)$$

$$\Rightarrow a^k b B^{k-1} C^k \quad (4)$$

$$\Rightarrow \dots \Rightarrow a^k b^k C^k \quad (5)$$

$$\Rightarrow a^k b^k c C^{k-1} \quad (6)$$

$$\Rightarrow \dots \Rightarrow a^k b^k c^k \quad (7)$$

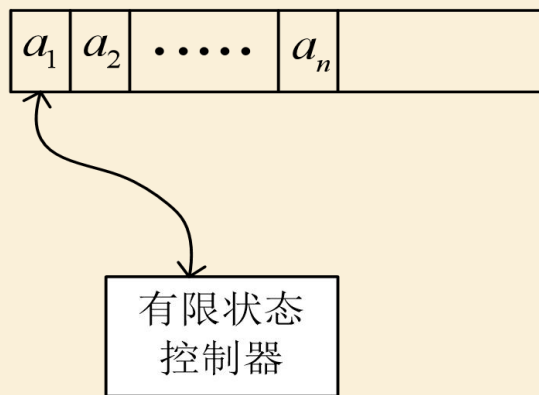
结论: CSG、CFG、正规式能力递减

但是: 能力越强的文法, 其文法的设计和自动机的构造越困难

因此: 语法分析仅用到CFG (除特别指出, 文法即指CFG)

有限自动机 (Finite Automaton, FA)

有限自动机是一种最简单的语言识别器，它也是有限状态系统的一种模型。它的基本结构如下图所示。



图有限自动机的基本结构

有限状态系统

在系统的运行过程中，系统的状态往往在不断变化。对许多实际系统中，整个运行过程出现的状态都只有有限多种。这样的系统称为有限状态系统。

确定的有限自动机

从这个图中找出从初始状态到目标状态的一条有向路，就是这个问题的一個解。

确定的有限自动机（deterministic finite automaton, DFA）

△ 基本定义

定义确定的有限自动机是一个五元组

$$M = (Q, \Sigma, \delta, q_0, F)$$

其中 Q 是有限状态集， Σ 是字母表， q_0 是初始状态（ $q_0 \in Q$ ）， F 是终止状态集（ $F \subseteq Q$ ）。 δ 称为状态转换函数，它是状态集 Q 与字母表 Σ 的笛卡尔积到状态集的一个映射，即

$$\delta: Q \times \Sigma \rightarrow Q$$

例 $M_1 = (Q, \Sigma, \delta, q_0, F)$

其中 $Q = \{q_0, q_1, q_2, q_3\}, \Sigma = \{0, 1\}, F = \{q_0\}$

不确定的有限自动机

不确定的有限自动机 (**nondeterministic finite automaton, NFA**)

△ **NFA** 的定义

定义 NFA 是一个五元组

$$M = (Q, \Sigma, \delta, q_0, F)$$

其中 Q, Σ, q_0, F 的含义同 DFA

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

2^Q 是集合 Q 的幂集 (也可用 $\rho(Q)$ 表示), 即由 Q 的各个子集组成的集合。

有限自动机 (Finite Automaton, FA)

许多问题都可以归结为有限状态系统。例如电子计算机中的编译程序是一个有限状态系统，操作系统是一个有限状态系统，CPU 是一个有限状态系统，整台电子计算机也是一个有限状态系统。

有一些问题，表面上看同“系统”似乎没有什么关系，其实也可以归结为有限状态系统来求解。

例如：狼、羊、菜问题。

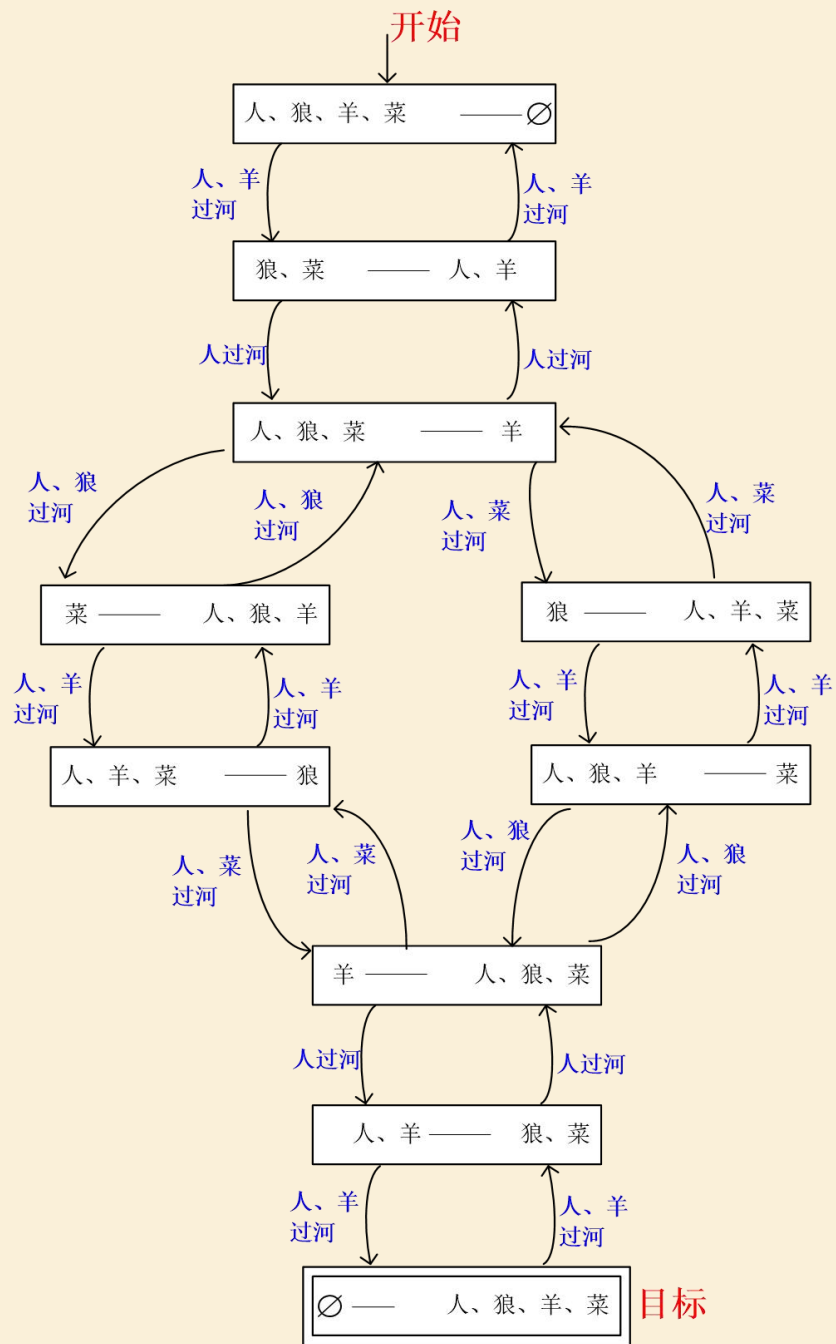
表面上看，这只是一个动脑筋的难题。如果用有限状态系统的思想方法来求解这个问题，就要方便得多。

刚开始时，人、狼、羊、菜都在河的左岸，我们把这种情况看作系统的初始状态。

最后，我们要求人、狼、羊、菜都到达河的对岸，这是目标状态。

由于小船的承载量有限，从初始状态不可能一步到达目标状态，需要有一些中间状态来过渡。而这些过渡是有条件的，过渡的条件就是人带着部分动物或菜过河，或者人单独过河。

下面的图给出了这个问题的所有可能出现的状态，以及状态的转换情况。



带输出的有限自动机

除了作为正规语言的识别器以外，有限自动机也常用作系统模型。这时，有限自动机的功能表现形式如下图



即对一个输入信号进行加工，产生一个输出信号（当然，还有 FA 内部的状态变化）。

介绍两种不同的带输出的有限自动机。

Moore机

定义 Moore 机是一个 6 元组

$$M = (Q, \Sigma, \Delta, \delta, \beta, q_0)$$

其中 Q, Σ, δ, q_0 的含义同 DFA, Δ 是输出字母表, β 是状态一输出函数。

即

$$\delta: Q \times \Sigma \rightarrow Q$$

$$\beta: Q \rightarrow \Delta$$

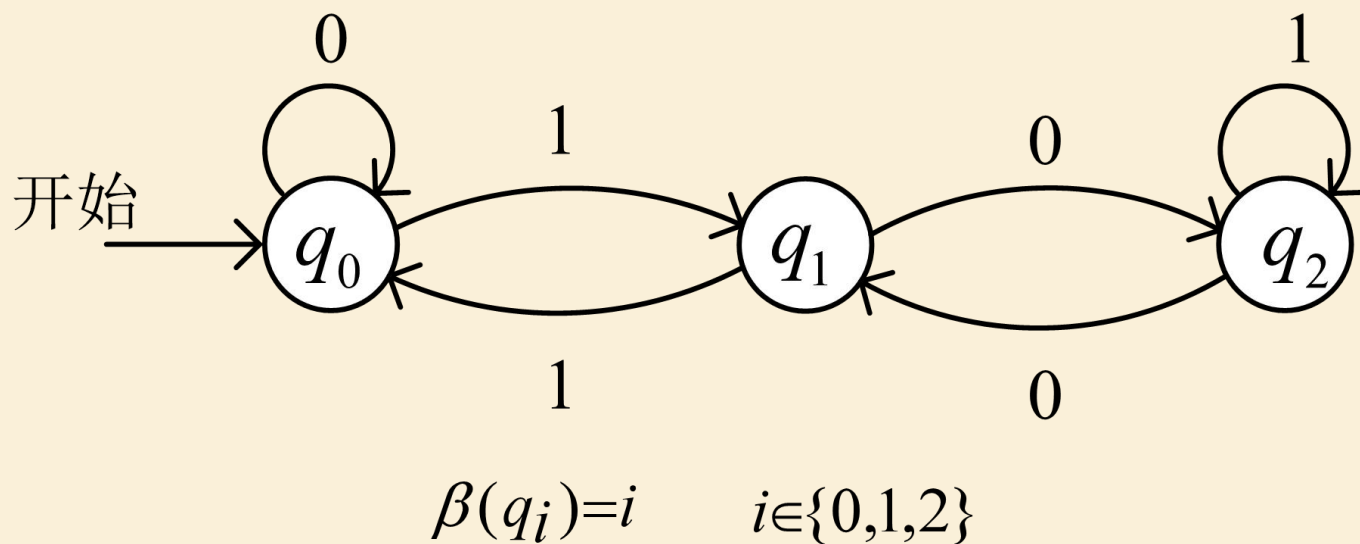
Moore 机中没有终止状态集, 而增加了输出字母表 Δ 和状态一输出函数 β 。下面通过例子说明怎样用 Moore 机构建系统模型。

例 构造一个 Moore 机 M , 对输入的任意二进制数 x , M 能计算 $x(\bmod 3)$, 并输出计算结果。

分析: 1) 显然 M 的输入字母表为 $\Sigma = \{0, 1\}$ 。由于一个数被 3 除的余数有 3 种可能 0, 1 或 2, 因此输出字母表为 $\Delta = \{0, 1, 2\}$ 。

Moore机

这个 Moore 机的图形表示如下（其中输出函数 β 无法在图中标出，写在图的下面）。



Mealy机

定义 Mealy 机是一个六元组

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

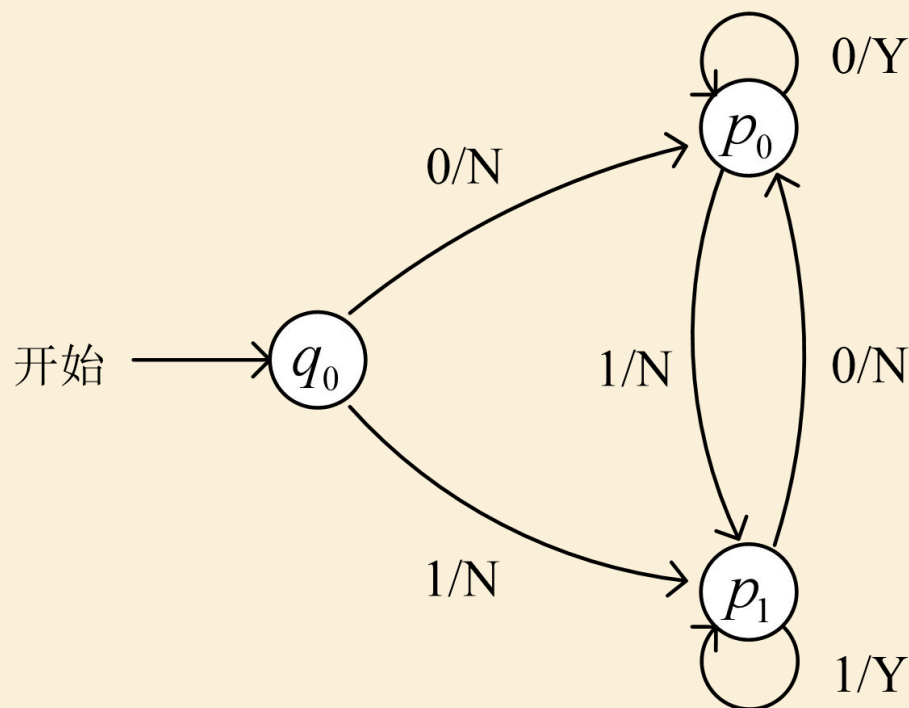
其中 $Q, \Sigma, \Delta, \delta, q_0$ 同 Moore 机, λ 是输出函数

$$\lambda: Q \times \Sigma \rightarrow \Delta$$

同 Moore 机相比较, Mealy 机是更为常见的一种系统模型。它的输出信号由系统的状态和输入信号共同决定。许多控制系统都属于这样的一种系统。因此, 许多控制系统的设计, 都是根据要求先设计出相应的 Mealy 机, 然后把 Mealy 机转化为逻辑电路。这里通过一个例子说明怎样构建实现某些功能要求的 Mealy 机。

Mealy机

这个 Mealy 机的图形表示如下



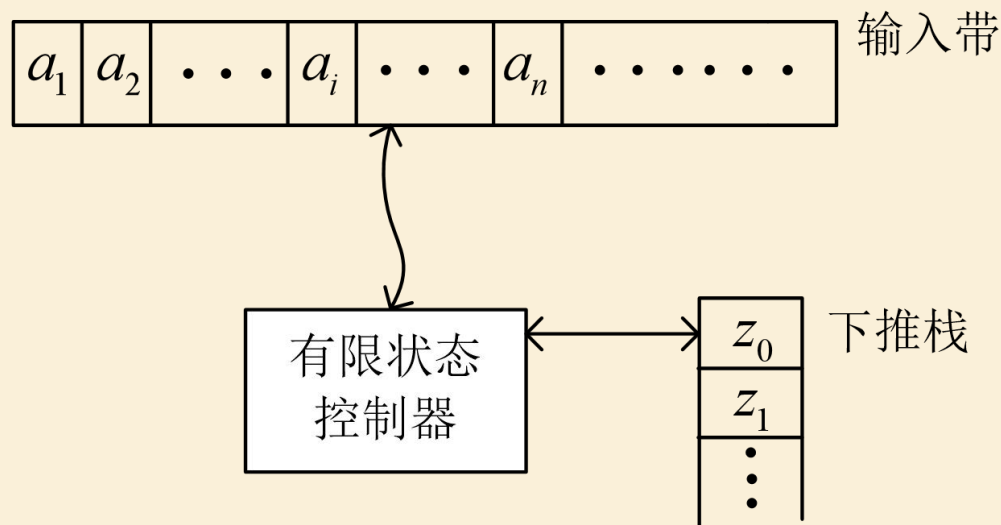
在 Mealy 机的图形表示中，每条有向边的标注都有两个字符，其中 “/” 左上方的为输入字符，右下方的为输出字符。

下推自动机 (Pushdown Automaton)

正如： 正规语言 $\leftrightarrow$ 有限自动机 一样

上下文无关语言 $\leftrightarrow$ 下推自动机

下推自动机的结构如下图所示：



下推自动机 (Pushdown Automaton)

下推自动机分为以空栈接受语言的下推自动机和以终止状态接受语言的下推自动机两种，它们的运行规则基本相同，只是在确定一个输入的字符串是否被接受的方式上有所不同。

定义 下推自动机 (PDA) 是一个七元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset) \quad (3.7)$$

或 $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F) \quad (3.8)$

其中 Q 是有限状态集， $q_0 \in Q$ 是初始状态， F (3.8 式) 是终止状态集 ($F \subseteq Q$)，(3.7) 式中的 $\emptyset$ 表示 M 是以空栈接受语言的下推自动机； Σ 是输入字母表， Γ 是下推栈字母表， $Z_0 \in \Gamma$ 是初始栈顶元素， δ 是状态与栈元素转换函数：

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*} \quad (3.9)$$

下推自动机 (Pushdown Automaton)

(3.9) 式表明, 下推自动机在某个状态 $q \in Q$ 和某个栈顶元素 $Z \in \Gamma$ 下, 不仅扫描到某个字符 $a \in \Sigma$ 时, 可以有动作

$$\delta(q, a, Z) = \{(q_1, r_1), (q_2, r_2), \dots, (q_k, r_k)\}$$

而且不扫描任何字符, 也可以有动作 (称为空动作)

$$\delta(q, \varepsilon, Z) = \{(q_1, r_1), (q_2, r_2), \dots, (q_k, r_k)\}$$

(3.9) 式还表明, 定义给出的是不确定的下推自动机 (NPDA) 的定义。今后就把它直接称为下推自动机 (PDA)。对于确定的下推自动机 (DPDA), 则不能省去“确定的”的定语。因为 NPDA 识别的语言类是全体 CFL 语言类, DPDA 则只能识别 CFL 语言类的一个子集。

无限制文法

定义 设 $G = (V, T; P, S)$ 为一个文法。如果对于 G 中的产生式：

$$\alpha \rightarrow \beta \quad \alpha, \beta \in (V \cup T)^*$$

除要求 α 中至少含有一个变量以外，没有其它任何限制，则称 G 为一个无限制文法。

也就是说，无限制文法中产生式 $\alpha \rightarrow \beta$ 的两端分别为

$$\alpha \in (V \cup T)^* V (V \cup T)^*$$

$$\beta \in (V \cup T)^*$$

这同第一章中给出的文法定义完全一样。

无限制文法也称为 0 型文法、半 Thue 文法或短语结构文法，其中 0 型文法的称呼使用最为广泛，后面还会专门来说明这种称呼的含义。

无限制文法

例 设 $G = (V, T; P, S)$, 其中

$$V = \{S, A, B, C, D, E\} \quad T = \{a\}$$

$$P: S \rightarrow ACaB \quad Ca \rightarrow aaC$$

$$CB \rightarrow DB \quad CB \rightarrow E$$

$$aD \rightarrow Da \quad AD \rightarrow AC$$

$$aE \rightarrow Ea \quad AE \rightarrow \varepsilon$$

从产生式的形式可以看出, G 是一个无限制文法。下面给出的是文法 G 的两个推导:

$$S \Rightarrow ACaB \Rightarrow AaaCB \Rightarrow AaaE \Rightarrow AaEa \Rightarrow AEaa \Rightarrow aa$$

$$S \Rightarrow ACaB \Rightarrow AaaCB \Rightarrow AaaDB \Rightarrow AaDaB \Rightarrow ADaaB$$

$$\Rightarrow ACaaB \Rightarrow AaaCaB \Rightarrow AaaaCCB \Rightarrow AaaaaE$$

$$\Rightarrow AaaaEa \Rightarrow AaaEaa \Rightarrow AaEaaa \Rightarrow AEaaaa \Rightarrow aaaa$$

图灵机

△ 图灵机在计算机科学中的地位

图灵机是：

- ☆ 自动机的最高形式
- ☆ 现代电子计算机的理论模型
- ☆ 可计算性和计算复杂性的判定基准

△ 图灵机概念提出的历史背景

- ☆ 希尔伯特纲领

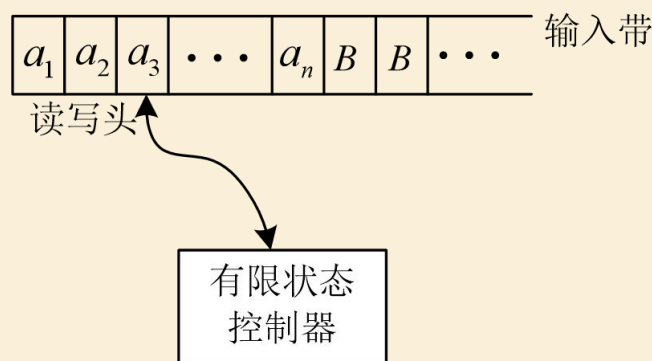
希尔伯特——19 世纪末 20 世纪初的世界领头数学家

数学基础问题：数学史上的三次数学危机

希尔伯特纲领的目的：一劳永逸地解决数学基础问题

图灵机

从机械装置的角度来说，图灵机由一条可以（向右）无限延长的输入带、一个有限状态控制装置和一个连接控制器与输入带的读写头组成，如下图所示



有限状态控制器的状态转换和动作规则是图灵机的核心要素。图灵机在当前状态下扫描到带上的一个字符时，要执行 3 件工作：

- 1) 进行状态转换；
- 2) 读写头在带上的当前格写上新的字符；
- 3) 决定读写头向左还是向右移动一格。

图灵机

对于带上的一个输入串，图灵机从初始状态和带上的最左格开始，通过连续不断地扫描带上的字符并执行相关的动作，如果在某个时刻进入终止状态，图灵机就接受这个输入串。被一个图灵机接受的全部字符串的集合就是这个图灵机所接受的语言。

定义 4.1 图灵机是一个七元组

$$M=(Q,\Sigma,\Gamma,\delta,q_0,B,F)$$

其中 Q 是有限状态集， $q_0(q_0 \in Q)$ 是初始状态， $F(F \subseteq Q)$ 是终止状态集； Γ 是带符号集， $B(B \in F)$ 表示空白符， $\Sigma(\Sigma \subseteq \Gamma - \{B\})$ 是输入字母表

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

称为动作函数（ L 表示读写头向左移一格， R 表示读写头向右移一格），对于某些状态 q 和某些字符 $a \in \Gamma, \delta(q, a)$ 可以没有定义。

图灵机

定义设 L 为一个语言，若存在一个图灵机 M 使得 $L = L(M)$ ，则称 L 为一个递归可枚举（recursively enumerable，简记为 r.e.）语言。

如果存在一个图灵机

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

使得 $L = L(M)$ ，且对任意输入串 $x \in \Sigma^*$ ， M 都停机，则称 L 为一个递归（recursive）语言。

显然，递归语言类是递归可枚举语言类的一个真子类。

上下文有关文法 (context-sensitive grammar, CSG)

定义 设 $G = (V, T; P, S)$ 为一个文法, 如果 G 中的产生式 $\alpha \rightarrow \beta$ 都满足 $|\alpha| \leq |\beta|$, 那么称 G 为一个上下文有关文法。

上下文有关文法产生的语言称为上下文有关语言 (context-sensitive language, CFL)

例 设 $G = (V, T; P, S)$, 其中 $V = \{S, A, B, C\}, T = \{a, b, c\}$,

P : ① $S \rightarrow aSBC$ ② $S \rightarrow aBC$ ③ $CB \rightarrow BC$

④ $aB \rightarrow ab$ ⑤ $bB \rightarrow bb$ ⑥ $bC \rightarrow bc$

⑦ $cC \rightarrow cc$

线性界限自动机 (Linear bounded automaton)

定义设 M 为一个不确定的图灵机，如果 M 满足条件

1) 输入字母表中包含两个特殊的带符号 $\$$ 和 ϵ ，分别为左端符和右端符；

2) M 在运行过程中，读写头只能在带上左端符 ϵ 和右端符 ϵ 之间的格上移动和写上新字符。

则称 M 为一个线性界限自动机 (LBA)。

一般地，一个线性界限自动机记为

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \epsilon, \epsilon, F)$$

它所接受的语言为

$$L(M) = \{w \in (\Sigma - \{\epsilon, \epsilon\})^* \mid q_0 \epsilon w \epsilon \xrightarrow[M]{*} \alpha q \beta, q \in F\}$$

各种类型的语言之间的关系

文法体系的四个类型

记文法 $G = (V, T; P, S)$, 4 种类型的文法的产生式分别为

$$\text{正规文法: } \begin{cases} A \rightarrow wB (\text{或} A \rightarrow Bw) \\ A \rightarrow w \end{cases} \quad A, B \in V, w \in T^* \quad (5.6)$$

$$\text{上下文无关文法: } A \rightarrow \beta \quad A \in V, \beta \in (V \cup T)^* \quad (5.7)$$

$$\begin{aligned} \text{上下文有关文法: } \alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2 \quad A \in V, \\ \alpha_1, \alpha_2 \in (V \cup T)^*, \beta \in (V \cup T)^+ \end{aligned} \quad (5.8)$$

$$\text{无限制文法: } \alpha \rightarrow \beta \quad \alpha \in (V \cup T)^* V (V \cup T)^*, \beta \in (V \cup T)^* \quad (5.9)$$

各种类型的语言之间的关系

正规语言类是上下文无关语言类的真子类；上下文无关语言类是上下文有关语言类的真子类；上下文有关语言类是递归可枚举语言类的真子类。

为了直观、简明地表述这 4 类语言之间的关系，人们用 0—3 型来分别记这 4 类文法和语言。即

0 型文法：无限制文法

1 型文法：上下文有关文法

2 型文法：上下文无关文法

3 型文法：正规文法

$i(i \in \{0, 1, 2, 3\})$ 型文法所产生的语言就称为 i 型语言。

各种类型的语言之间的关系

